

□ Desafío Integrador: Motor de Personajes RPG (Videojuego) - Guía Paso a Paso

Objetivo: Convertirte en el Arquitecto de Software del núcleo de un nuevo videojuego. Deberás modelar la clase `Personaje`, asegurando que sea imposible hacer "trampa" (hackear sus estadísticas) gracias a un uso estricto del Encapsulamiento, constructores seguros y la Programación Defensiva.

Paso 1: Estado Interno y Memoria Compartida (Atributos)

Todo gran objeto comienza con un estado blindado. Debes declarar todos los atributos como estrictamente privados. Además, agregaremos un atributo estático (que pertenece a la clase y no al objeto) para llevar la cuenta de los personajes creados y asignarles un ID único.

Tu tarea:

- Crea la clase `Personaje`.
- Declara los atributos privados: `nombre (String)`, `nivel (int)`, `puntosVida (int)` e `isDerrotado (boolean)`.
- Agrega los atributos para la identidad: `id (int)` y `totalPersonajes (static int)`.

Código de ayuda (Generador de ID):

Copia y pega este método privado dentro de tu clase. Se encargará de usar la variable estática para darte un número de identificación irrepetible.

```
// Atributo estático compartido por toda la clase
private static int totalPersonajes = 0;

// Método utilitario interno para auto-incrementar
private int generarId() {
    totalPersonajes++;
    return totalPersonajes;
}
```

Paso 2: El Escudo Protector (Getters y Smart Setters)

No podemos dejar que cualquier usuario asigne un nivel negativo o una vida infinita. Implementa los métodos `get` para todos los atributos. Luego, crea Smart Setters que validen la información. Si el dato es inválido, recházalo y muestra un error por consola.

Reglas de validación:

- Nombre: No puede ser nulo ni estar vacío.
- Nivel: Rango permitido de 1 a 100.

- Vida: Rango permitido de 50 a 100.
- isDerrotado: No lleva setter. Nace en `false` y solo cambia si la vida llega a 0.

Código de ayuda (Ejemplo de Smart Setter para Nivel):

```
public void setNivel(int nivel) {
    if (nivel >= 1 && nivel <= 100) {
        this.nivel = nivel; // Usamos 'this' para desambiguar
    } else {
        System.out.println("Error: El nivel debe estar entre 1 y 100. Se mantiene el nivel actual.");
    }
}
```

¡Ahora te toca a ti armar los setters para el nombre y los puntos de vida!

Paso 3: El Contrato de Nacimiento (Constructores y Sobrecarga)

Un personaje no puede nacer con datos en blanco. Crearemos constructores que obliguen al usuario a dar los datos iniciales, y usaremos nuestros propios Smart Setters dentro del constructor para asegurar que nazcan validados.

Tu tarea:

- Crea un constructor completo que reciba `nombre`, `nivel` y `puntosVida`.
- Crea un constructor parcial (sobrecarga) que reciba solo el `nombre`, y encadénalo al principal usando `this(...)`.

Código de ayuda:

```
// Constructor Parcial (Usa this() para aplicar el principio DRY)
public Personaje(String nombre) {
    this(nombre, 1, 100); // Delega la construcción enviando nivel 1 y vida 100 por defecto
}

// Constructor Principal (Hace el trabajo real)
public Personaje(String nombre, int nivel, int puntosVida) {
    this.id = generarId(); // Asignamos el ID único automáticamente
    this.isDerrotado = false; // Estado inicial por defecto

    // ¡Usamos los patovicas (setters) para nacer de forma segura!
    this.setNombre(nombre);
    this.setNivel(nivel);
    this.setPuntosVida(puntosVida);
}
```

Paso 4: Comportamiento Defensivo (Métodos de Acción)

El personaje debe defenderse solo. Crea los siguientes métodos aplicando las reglas de negocio:

1. `recibirDanio(int danio)`: Resta vida. Si el daño supera la vida actual, la vida queda exactamente en 0 (nunca en negativo) y el personaje pasa a estado "Derrotado" (`isDerrotado = true`).
2. `curar(int puntos)`: Suma vida. Solo funciona si la vida actual es mayor a 0. La curación tiene un tope máximo de 100 puntos.

Paso 5: Identidad y Diagnóstico (`toString`, `equals`, `hashCode`)

Para poder ver el estado de nuestros personajes y compararlos correctamente, sobreescribiremos los métodos heredados de la clase `Object`.

Tu tarea:

- Genera el método `toString()` (Tu "carnet" de personaje) que muestre el ID, Nombre, Nivel, Vida y si está derrotado. Esto reemplaza al antiguo `mostrarEstado()`.
- Genera `equals()` y `hashCode()` utilizando únicamente el atributo `id`. Dos personajes son el mismo solo si su ID coincide en el sistema. Puedes usar la herramienta de tu IDE (Click derecho -> Generar) para hacer esto rápido y sin errores matemáticos.

□ Paso 6: Misión en la Clase Main (Prueba de Robustez)

Tu método `main` actuará como un Tester de Calidad (QA) intentando romper el sistema. Sigue este guion de pruebas:

Fase 1: Creación

Instancia a `guerrero` ("Thor", nivel 5, vida 100) y a `magico` ("Gandalf", nivel 10, vida 50). Imprime ambos objetos en consola (¡tu `toString()` hará la magia!).

Fase 2: Hackeando el Sistema (Pruebas de Fallo)

- Intenta setearle a Thor un nivel de -5.
- Intenta curar a Gandalf 200 puntos.
- *Revisa la consola: Tus Smart Setters deberían haber bloqueado ambos ataques.*

Fase 3: Simulación de Batalla

- Haz que Thor reciba 150 de daño. Su vida debe quedar en 0 y quedar derrotado.
- Intenta curar a Thor con 20 puntos. El sistema debe rechazarlo porque está derrotado.
- Imprime ambos objetos nuevamente.